



Programação web

Introdução ao JS - Parte 2

Prof. Diego Cardoso Borda Castro



Objetos

- Conjunto de propriedades atribuídas à uma variável
- Podem representar objetos da vida real



```
1  let variavel = {  
2      chave1: valor1,  
3      chave2: valor2,  
4      chave3: valor3,  
5      ...  
6      chaveN: valorN  
7  };
```



Objetos

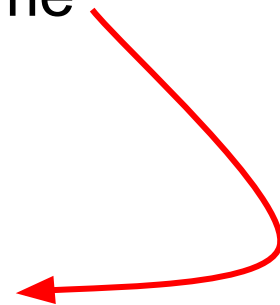
- `let professor = { nome: "Diego", idade: 19 };`
 - `console.log(professor.nome);`
 - `console.log(professor['nome']);`
 - Semelhante a como é referido um valor dentro de um array
 - `professor.sobrenome = 'Cardoso';`
 - `console.log(professor.nomeCompleto);`
 - `undefined`

Objetos

- `console.log(professor.nomeCompleto);`
 - `professor.nome + " " + professor.sobrenome`



```
1  const professor = {  
2      nome: "Diego",  
3      sobrenome: "Cardoso",  
4      nomeCompleto: function () {  
5          return this.nome + " " + this.sobrenome;  
6      }  
7  };
```





Objetos

- `let obj = { nome: "Diego", idade: 19 };`
 - `console.log(Object.keys(obj));`
 - `["nome", "idade"]`
 - `console.log(Object.values(obj));`
 - `["Diego", 19]`
 - `let newObj = Object.assign({}, obj, { funcao: "professor" });`
 - `{ nome: "Diego", idade: 19, funcao: "professor" }`



Loop

- Repetição

- Fazer a mesma coisa repetidas vezes

```
1 while(expressao) {  
2     // código a ser executado  
3 }
```

- While

- Executa comandos toda vez que uma expressão lógica especificada é verdadeira
- Condicional
 - Expressão
 - True
 - False



Loop

- Do while
 - Conceito igual ao while
 - Condição **DO** sempre será executada pelo menos uma vez

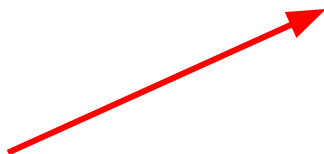
```
1 do {  
2     // código a ser executado  
3 } while(expressao);
```

```
let obj = {  
    nome: "Diego",  
    sobrenome: "Cardoso",  
    idade: 19  
};  
  
let cont = 0;  
let lista = Object.keys(obj);  
let values = Object.values(obj);  
  
while( cont < lista.length){  
    console.log(values[cont]);  
    cont++;  
}
```



Loop

- For
 - Inicializacao
 - Expressao
 - Incremento
- Expressões opcionais



```
1 let values = Object.values(obj);
2
3 for(let cont = 0; Object.keys(obj).length > cont ;cont++){
4     console.log(values[cont]);
5 }
```

```
1 let obj = {
2     nome: "Diego",
3     sobrenome: "Cardoso",
4     idade: 19
5 };
6
7 let cont = 0;
8 let lista = Object.keys(obj);
9 let values = Object.values(obj);
10
11 for(;;cont++){
12     if(values.length == cont)
13         break;
14     console.log(values[cont]);
15 }
```




Tratativa de erros

- Seu código vai quebrar!
- Asseguram qualquer tipo de exceção
 - Try / Catch
 - Throw
- Objeto Erro
 - ReferenceError
 - SyntaxError
 - RangeError
 - InternalError
 - EvalError
 - Custom error



Tratativa de erros

- Try
 - Onde um bloco de código é executável
- Catch
 - É executado em caso de erro
- Finally
 - Sempre será executado no final

```
1  try {  
2      // seu código aqui  
3  } catch (error) {  
4      // tratamento de erro aqui  
5  } finally {  
6      // executa sempre  
7  }
```



Tratativa de erros

- Erros nativos
 - Não temos ideia do erro que pode acontecer
- Erros customizados
 - Regras de negócio bem definidas
 - Padrão de erros

```
1 class MeuErro extends Error {  
2     constructor(msg) {  
3         super(msg);  
4         this.name = 'MeuErro';  
5     }  
6 }  
7  
8 try {  
9     throw new MeuErro("Mensagem do meu erro!")  
10 } catch (error) {  
11     console.log(error.name)  
12     console.log(error.message)  
13 }  
14 }
```



Tratativa de erros

- Throw
 - Lançar uma exceção
- Try / Catch
 - Capturar o erro lançado
- Erro Customizado
 - Regras de negócio

```
1  throw "Descrição do erro";
2
3  throw 404;
4
5  throw {
6      name: 'Erro',
7      message: 'Descrição do erro'
8  }
9
10 throw new UserTypeError('Tipo de usuário inválido');
```



Módulo e Modularidade

- **Esconder** detalhes de implementação
- Permitir que aplicações maiores sejam montadas de forma **modular**
- CJS (Common JS)
 - Primeira forma de modularidade
 - Objeto global exports do Node.js
 - **require()**



Módulo e Modularidade

- Por não existir um método próprio
 - ES6 especificou
 - EcmaScript Modules
 - Import
 - export

```
1 const { soma, multiplica } = require('./operacoes.js');
```

```
1 import validaCartao from 'validacoes/validacaoCartao.js';
```



Módulo e Modularidade

```
1 function soma(num1, num2) {  
2     return num1 + num2;  
3 }  
4  
5  
6 function multiplica(num1, num2) {  
7     return num1 * num2;  
8 }  
9
```

```
1 import funcoes from './arquivo.js';  
2 let a = funcoes.bolinha(1, 2);
```

```
1 function subtrai(num1, num2) {  
2     return num1 - num2;  
3 }  
4  
5  
6 module.exports = soma;
```

```
1 export default {  
2     soma: soma,  
3     subtrai: bolinha,  
4     multiplica: multi  
5 }  
6
```



Módulo e Modularidade

- Módulos dentro de bibliotecas
 - `import { body } from 'express-validator';`
 - **Importa apenas o módulo que está dentro da lib**
- Para utilizar o ESM em aplicações Node.js
 - Adicionar a propriedade

```
1 {  
2   "name": "Cefet-API",  
3   "version": "1.0.0",  
4   "description": "API do cefet",  
5   "main": "index.js",  
6   "type": "module",  
7   "author": "Diego",  
8   "license": "ISC"  
9 }
```




Módulo e Modularidade

```
1
2 function validaCartao(cartaoRecebido) {
3     const resultado = funcaoAuxiliar(cartaoRecebido)
4     return resultado
5 }
6
7 export default validaCartao;
```

```
1 export default {
2     soma: soma,
3     subtrai: bolinha,
4     multiplica: multi
5 }
6
```

```
1 export function minhaFuncao() {
2     console.log("Função exportada");
3 }
```

```
1 export { minhaFuncao as novaFuncao };4
```

- Export default
 - Importa com qualquer nome
- Export
 - Importa com nome que foi exportado



Módulo e Modularidade

- CJS funciona de forma síncrona
- ESM funciona de forma assíncrona

```
1 import soma from './math.js';
2
3 import soma, { subtrai } from './math.js';
4
5 import { soma as add } from './math.js';
6
7 import * as math from './math.js';
8
9 const soma = require('./math');
10
11 const { soma, subtrai } = require('./math');
12
```



Paradigmas de programação

- Imperativo
 - Afirmações para alterar o estado da aplicação
 - Modo verbal imperativo
 - Ordem para executar algo
 - Tipos
 - Estrutural
 - Procedural
 - Orientação a objetos



Paradigmas de programação

- Imperativo
 - Como percorrer o vetor
 - Qual operação fazer
 - O que devemos adicionar ao resultado

```
1  function dobra(vetor){  
2      let resultados = [];  
3      for (let i = 0; i < vetor.length ; i++){  
4          resultados.push(arr[i] * 2);  
5      }  
6      return resultados;  
7  }  
8
```



Paradigmas de programação

- Declarativo
 - Expressar a lógica de um processo sem descrever o seu controle de fluxo
 - Associado a **o quê** uma tarefa vai resultar ou retornar
 - Tipos
 - SQL
 - Funcional

```
1  function dobra(vetor){  
2      return vetor.map((item) => item * 2);  
3  }  
4
```

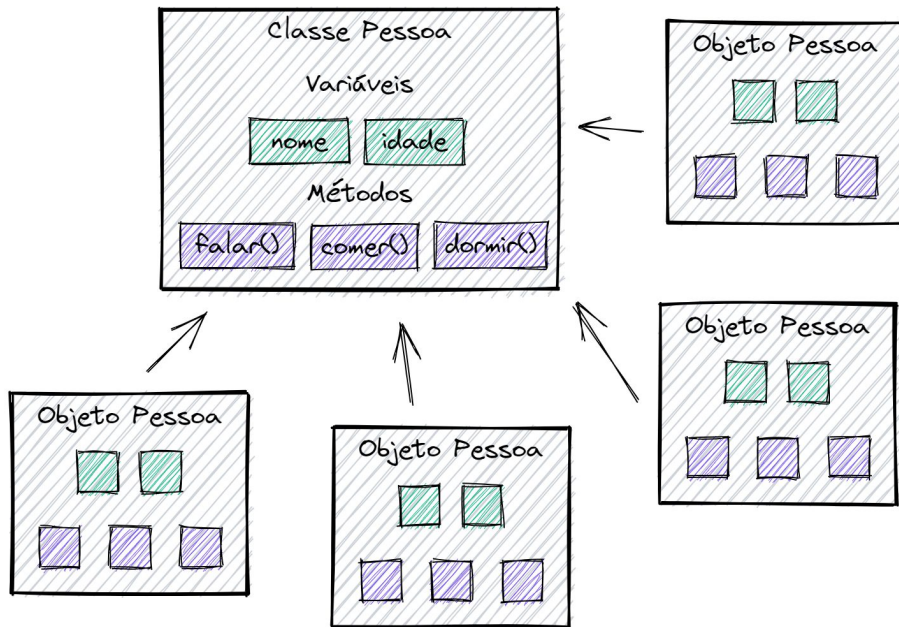


Paradigmas de programação

- JavaScript
 - Multiparadigma
 - Perfis diferentes de desenvolvedores e sistemas utilizarem uma linguagem em comum
 - Não tem um que seja melhor que o outro
 - Existe o mais adequado para o problema

Orientação a objetos

- Modelo para a criação de objetos
- Orientação a objetos
- Mesmas propriedades de um objeto para reutilizar
- Base para gerar cópias





Orientação a objetos

- Construtor
- Get
- Setter
- Método
- Instância

```
1  class Aluno {  
2      constructor(matricula, nome) {  
3          this.matricula = matricula;  
4          this.nome = nome;  
5      }  
6  
7      get matricula() {  
8          return this.matricula;  
9      }  
10  
11     set matricula(matricula) {  
12         return this.matricula = matricula;  
13     }  
14  
15     trancar() {  
16         console.log("Matrícula trancada!");  
17     }  
18 }  
19  
20 let aluno1 = new Aluno("123", "Diego");  
21 console.log(aluno1.matricular());
```




Funções

- Função tradicional
- Função inline
- Arrow function



```
1 function soma(num1, num2) {  
2     return num1 + num2;  
3 }  
4
```



```
1 const soma = function(num1, num2) {  
2     return num1 + num2;  
3 }  
4
```



```
1 const soma = (num1, num2) => {  
2     return num1 + num2;  
3 }  
4
```



```
1 const soma = (num1, num2) => num1 + num2;
```



Async / await

- Realização de uma atividade em **segundo plano**
- Sem nosso controle direto
- Sem trabalhar com threads e coordená-las
- Síncrona
 - Telefone
- Assíncrona
 - Whasapp
 - Promise
 - Promise
 - Resolve
 - Reject



Async / await

- Promises
 - Then
 - Callback
 - Objeto de promessa

Não é um retorno dos dados, é a promessa do retorno destes dados.

```
1 function getUser(userId) {  
2     const userData = fetch(`https://api.com/api/user/${userId}`)  
3         .then(response => response.json())  
4         .then(data => console.log(data.name))  
5 }  
6  
7 getUser(1);
```



Async / await

```
1 function getUser(userId) {  
2     const userData = fetch(`https://api.com/api/user/${userId}`)  
3     .then(response => response.json())  
4     .then(data => console.log(data.name))  
5     .catch(error => console.log(error))  
6     .finally(() => console.group("Terminou"))  
7 }  
8
```

- Promise.all
 - Executa todas as promessas de uma vez
 - Muito usado p/ BDs

```
1 const endpoints = [  
2     "https://api.com/api/user/1",  
3     "https://api.com/api/user/2",  
4     "https://api.com/api/user/3",  
5     "https://api.com/api/user/4"  
6 ]  
7  
8 const promises = endpoints.map(url => fetch(url).then(res => res.json()))  
9  
10 Promise.all(promises)  
11     .then(body => console.log(body.name))
```



Async / await

- Simplificação das atividades assíncronas
 - Async
 - Await
 - Funciona como assíncrono
 - Esconde as premissas
 - Lida como síncrona
- Síncrona
 - Telefone
- Assíncrona
 - Whasapp
 - Promise
 - Promessa de que algo será feito



Async / await

- Async
 - Funções assíncronas
 - Retornam Promises
 - Sem o valor real de retorno
 - Espera a resposta
- Await
 - Transforma funções assíncronas em síncronas
 - Transforma promises em retornas das chamadas

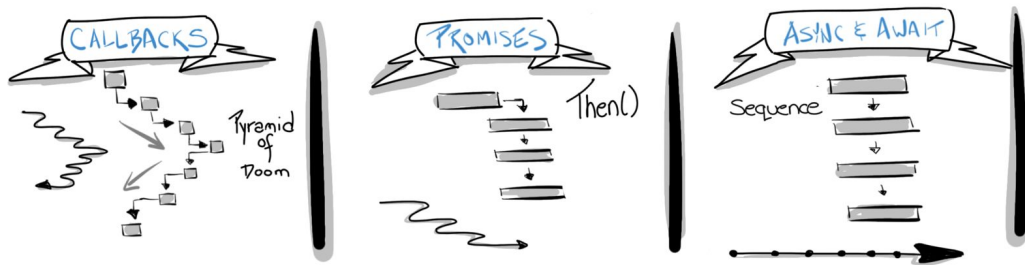
```
1 let response = await fetch(`https://api.com/api/user/${userId}`);
```



Async / await

- Sintaxe
 - Then
 - JavaScript funcional
 - Async/await
 - Métodos e classes

Asynchronous TypeScript





Exercícios

- <https://replit.com/languages/nodejs>
- Crie uma função que divida dois números e trate um erro se a divisão for por zero



Exercícios

- <https://replit.com/languages/nodejs>
- Crie um sistema simples de gerenciamento de produtos
 - Produtos.js
 - Array de produto
 - Lista os produtos
 - Gerenciamento.js
 - Adicionar produtos
 - Remover produtos
 - Splice
 - Index.js
 - Chama as funções para testar